Code Jam 2022 - Round 3

Analysis: Duck, Duck, Geese

Test Set 1

Given the lower limits for $\mathbf{N}$, we can check all possible contiguous subsets (as long as we check each one quickly). One way is to iterate through all possible starting indices for the contiguous subset. Then, for each starting index, we can loop through all possible sizes in order.

We can keep track of the counts of each color as well as the number of hat colors have counts that are invalid (not $\mathbf{0}$ and not in the acceptable range for that color). Note that when we extend our subset, only the hat color that the newly added child has is affected. If this changed whether or not this color is valid, we update the count of valid and invalid colors as needed. Then, if this count is equal to $\mathbf{C}$, we can increment our total answer (as long as the subset is of length at least 2 and at most $\mathbf{N} - 1$).

Because checking each contiguous subset is done in $O(1)$ time and there are $O(\mathbf{N}^2)$ subsets to check, our time complexity for Test Set 1 is $O(\mathbf{N}^2)$.

Test Set 2

For Test Set 2, $\mathbf{N}$ is too large to check each contiguous subset separately. We can speed this up by using a Segment Tree to check all possible subset lengths at once (for each starting index). We can remove the cyclic part of the problem by appending the array to itself.

Given a starting index, S, each color has two (possibly empty) ranges of ending indices that are valid for this color (meaning the number of hats of this color is either 0 or in the acceptable range). If we use add 1 to our segment tree for each value in these ranges and for all colors, we can count the number values (ending indices) in our segment tree for the range $[S + 1, S + \mathbf{N} - 2]$ that have a value of exactly $\mathbf{N}$. This count will tell us how many contiguous subsets are valid for our current start index.

Now, all we need to do is update our segment tree when moving the starting index to the right. Notice that moving our starting index to the right by one will only affect the valid end index ranges for the hat color of the child being removed (the one that previously was our starting index). If we precompute for each position, where is the next occurrence of that child's hat color, we can compute how the valid ranges for this color will move in $O(1)$. The exact implementation of this is left as an exercise to the reader.

Given that our segment tree operations each take $O(\log \mathbf{N})$ time, we can count the number of contiguous subsets for each starting index and move the starting index to the right both in $O(\log \mathbf{N})$ time. This gives us a final time complexity of $O(\mathbf{N} \log \mathbf{N})$.

Code Jam 2022 - Round 3

Analysis: Mascot Maze

This is a variation of the classic graph coloring problem, where the rooms, exits, and mascots of the problem form the nodes, edges, and colors of a graph.

Cases where there is a cycle of length two (two rooms x and y where there is an exit from x to y and an exit from y to x) are obviously impossible. We will show later that these are the only impossible cases, by giving an algorithm that always produces a coloring for graphs with no cycles of length two.

Test Set 1

Given the low limits, it is possible to implement a backtracking solution: recursively try coloring each vertex in each color. There are techniques which will help speed up the solution. For example, you can keep track of the colors that shouldn't be used for a vertex (since one of its neighbors was previously colored with that color).

However, plain backtracking algorithms are unlikely to work for Test Set 2, since early color assignments might make it impossible to color later nodes, and it can take a long time until those early assignments are revisited by the algorithm.

Test Set 2

Consider the graph G' which has an edge from one node to another if there is a path of length 1 or 2 linking them in G. That is, G' contains the edge (v,w) if (v,w) is in G, or there is a pair of edges (v,x) and (x,w) in G. It is sufficient to color G' so that no adjacent pair of nodes has the same color.

Since each node in G' has outdegree at most 6 (2 nodes that are one edge away in G and 4 nodes that are two edges away in G), the average indegree must be at most 6, and so at least some individual node V must have degree at most 6 + 6 = 12. No matter what colors were chosen for this node's neighbors, there is always at least one different color for this node because it has at most 12 neighbors.

The only case when the coloring is impossible for G' is when it contains a self-loop. In such a case it is also impossible for G.

Using these observations the following algorithm can be used to color G':

1. Find some V with the degree not greater than 12.
2. Temporarily remove V (and the edges connected to it) from the graph.
3. Recursively color the rest of the graph (which still maintains all the described properties).
4. Reinsert V and color it.

Here is one way to implement this solution efficiently:

- Build an adjacency list and an array of degrees for G'. Make sure to track both incoming and outgoing edges.
- Run a breadth-first search to remove the vertices of degree not greater than 12 one by one. Start by adding all such vertices to the queue and use adjacency lists for efficient removal. If the degree of any neighbor of the vertex currently being removed becomes 12, add this vertex to the queue. Note that you don't need to remove the vertex from the

adjacency lists (as you will need those further), just update the degree. While running BFS keep track of the order in which the vertices are traversed.
- Go through the vertices in the reverse order and color each one greedily: try each color and use it if none of the neighbours still have it.

Every step of the algorithm can be done in linear time, so the overall time complexity of the solution is linear.

Some heuristic approaches, like local search, can also be made to work if implemented efficiently.

Code Jam 2022 - Round 3

Analysis: Revenge of GoroSort

In the original GoroSort problem from 2011, Goro could hold as many elements in place as he wanted; using the terminology of this problem, he could create arbitrary many box color groups with 1 ball (element) each. But unlike in this problem, he was only allowed to use at most one color group of size greater than 1. That made the optimal strategy relatively straightforward: he could repeatedly permute all list elements that were not in the correct places.

That strategy puts one additional element in the right place each turn, in expectation, which is clearly too slow for this problem. (Our version of that solution takes almost 100000 rounds!) What is more surprising is that the crux of the original problem — namely, thinking in terms of expected numbers of elements that become correctly placed after the bump — can even be *misleading* in this problem!

The perils of pairing

It is intutively clear that any elements that are already in the right place should be left alone (i.e., each should be put in its own color group). It also seems reasonable to not break up pairs of elements that are swapped (i.e., each is in the other's place). We can also reasonably guess that splitting such a pair would be useless, and that putting additional elements in that group would be worse than handling those other elements in their own group(s).

What about the other elements? One very tempting strategy is to try to pair them up into "trespasser pairs" such that one member of the group is in the other member's place. It may not be possible to do this with every element, but we can, e.g., find a way to tack leftovers onto existing trespasser pairs.

Consider a trespasser pair (x, y), where x belongs where y is and y belongs somewhere else entirely. The judge's "bumping" (permutation) process will put x in the right place with probability $\frac{1}{2}$. So if we somehow manage to create around $\frac{\mathbf{N}}{2}$ trespasser pairs, each bump will put around $\frac{\mathbf{N}}{4}$ more elements in the right places. This seems quite good, and it is good enough to pass Test Set 1, but not the other two. (Our trespasser-pair-based solutions take between 15000 and 16300 rounds, depending on the care taken with implementation details.)

Thinking in terms of cycles

Any permutation — and therefore any state in this problem — can be described as a multiset of cycles of particular lengths. For example, each element that is already in the correct place is a 1-cycle, and each pair of swapped elements is a 2-cycle.

If we try to implement the pairing idea above in a greedy way, we may miss some opportunities to form trespasser pairs. We can get as many as possible by first finding all the cycles, then going through each cycle and chopping it into trespasser pairs, plus perhaps one "trespasser trio" at the end. (A trespasser trio is a set of elements x, y, z, in some order, such that y is in x's place, z is in y's place, and z belongs outside of the group.) These improvements (especially creating trespasser trios where needed) help a lot, but not enough to pass Test Set 2; our trespasser pairs + one trio solution takes around 13100 rounds.

But what if we leave the cycles as they are, and make each cycle its own permutation group? Now we do much better, and pass Test Set 2 as well (but not Test 3; we take about 11800 rounds). Why is this cycle-based strategy so much better than the trespasser pair strategies?

When expectation doesn't satisfy our expectations

Here's the problem with chopping into trespasser pairs. Suppose that we have a 4-cycle like 2341. If we split it into two trespasser pairs 23 and 41, we will (in expectation) put one element in the correct place. But, as we will see later on, we get the same expectation of 1 if we designate the entire cycle as one group.

Does this necessarily mean these strategies are equally good? Let's set our expectations appropriately! We really care about the expected *total* number of rounds to finish sorting, so let's work toward calculating that instead. First we observe that:

- **If we split the 4-cycle into two trespasser pairs**:
  - With probability $\frac{1}{4}$, both elements end up in their correct places in the group, and we get two 1-cycles and one 2-cycle.
  - With probability $\frac{1}{4}$, neither element ends up in its correct place in the group, and we end up stuck at a 4-cycle.
  - Otherwise, with probability $\frac{1}{2}$, we end up with a 1-cycle and a 3-cycle.
- If we don't split the cycle before permuting:
  - With probability $\frac{1}{24}$, we get four 1-cycles.
  - With probability $\frac{1}{4}$, we get two 1-cycles and a 2-cycle.
  - With probability $\frac{1}{3}$, we get a 3-cycle and a 1-cycle.
  - With probability $\frac{1}{8}$, we get two 2-cycles.
  - With probability $\frac{1}{4}$, we are stuck at a 4-cycle.

Comparing these two probability distributions directly, and canceling out the parts that are the same, we need to know which is better:

- a $\frac{1}{6}$ probability of getting a 3-cycle and a 1-cycle, or
- a $\frac{1}{8}$ probability of getting two 2-cycles and a $\frac{1}{24}$ probability of getting four 1-cycles

Now we can assess each of those states in terms of the expected number of rounds to completion. A 3-cycle turns out to take 3 rounds, in expectation, to become all 1-cycles. (This comes from solving $\mathbf{E}[3] = 1 + \frac{1}{3} \cdot \mathbf{E}[3] + \frac{1}{2}\mathbf{E}[2] + \frac{1}{6}(0)$, and using the fact that $\mathbf{E}[2] = 2$.) By a similar analysis, two 2-cycles take $\frac{8}{3}$ rounds, in expectation, to become all 1-cycles. (And if we are lucky enough to reach four 1-cycles directly, 0 additional rounds are needed.)

Therefore, chopping up a 4-cycle into two trespasser pairs is strictly worse than leaving it as is! We wouldn't have known this if we had argued purely based on the expected number of correctly placed elements; it also matters what we leave behind. Intuitively, the trespassers can only be dealt with after their companions have been correctly placed, so the chopping strategy is less parallelizable. But we shouldn't assume this will always be true no matter how we chop...

Chopping is not always bad

It's tedious to perform the above analysis for cycles longer than 4. But we shouldn't give up hope. The expectations for the two strategies were the same for 4-cycles, but will that hold up in general?

Let's think about how many cycles we expect to see in a random permutation of length $\mathbf{N}$. You may have heard of the following problem: everyone loses their hats all at once, and each person puts on a random hat; in expectation, how many people get their own hats back? The probability

that the each person gets their own hat is $\frac{1}{\mathbf{N}}$, and then by linearity of expectation, the total number of instances of someone getting their own hat is $\frac{1}{\mathbf{N}} \cdot \mathbf{N} = 1$.

So this gives us the expected number of 1-cycles. We can make a similar argument about 2-cycles: there are $\binom{\mathbf{N}}{2}$ distinct pairs of elements that could form a 2-cycle, and for each one, the probability that each has the other's element is $\frac{1}{\mathbf{N}} \cdot \frac{1}{\mathbf{N}-1}$. This all boils down to $\frac{1}{2}$. Indeed, the answer for general k-cycles is $\frac{1}{k}$.

Then how many total cycles should we expect? It's $\frac{1}{1} + \frac{1}{2} + \ldots + \frac{1}{\mathbf{N}}$. You may recognize this as the expression for the $\mathbf{N}$-th harmonic number. The harmonic numbers grow rather slowly; H_{100}, for example, is just over 5.

Therefore, if there are only around 5 cycles in our initial random permutation of length $\mathbf{N} = 100$, we expect to place around 5 elements. But if we chop the cycles into, say, 50 trespasser pairs, we should expect to place around 25 elements! How can it possibly be better to leave such large cycles alone? Does the argument about leaving more mess behind really still hold up?

One issue with chopping into pairs is that, intuitively, many roads to completion include forming one or more 4-cycles, and we have now seen that the pair-chopping strategy mishandles them. (And now we have reason to suspect that it mishandles larger cycles as well).

What if we chop a cycle into chunks larger than pairs? For example, if we chop a 6-cycle into two trespasser trios, each of them will produce $2 \cdot \frac{1}{3} = \frac{2}{3}$ correctly placed elements in expectation. That's $\frac{4}{3}$ overall, which is less than the $3 \cdot \frac{1}{2} = \frac{3}{2}$ we would have gotten from chopping into trespasser pairs. But these trespasser trios leave less of a mess; each one can only leave one element to clean up later, so there will be two such elements as compared to three.

It is hard to directly weigh the costs of cleaning up these leftover elements against the benefits of correctly placing more elements in expectation. The math for our 4-cycle example was already a bit cumbersome to carry out in a timed round, and there is not a strong incentive to do even more of it when the local testing tool can quickly tell us how well a strategy does, and when all three test sets are Visible. Indeed, we can find that leaving all cycles of length less than 6 intact, and breaking all cycles of length 6 or more into trespasser trios (or trespasser sets of four or five, when there are extra elements) does much better than leaving all cycles intact, and lets us pass Test Set 3, taking about 10950 rounds.

We can do even better by choosing different chunk lengths (e.g. break each cycle of length c into chunks of size roughly $\sqrt{c}$, to get down to around 10500 rounds), but that is not necessary for this problem.

Code Jam 2022 - Round 3

Analysis: Win As Second

Introduction

This problem describes an impartial game which can be analyzed using the Sprague-Grundy theorem.

However, the number of possible states in the game is quite big. There can be $2^{\mathbf{N}}$ sets of blue vertices, and even after we use the Sprague-Grundy theorem to reduce the problem to consider only connected sets of vertices, the number of states is still big. For example, a star which has the center vertex connected to $\mathbf{N}-1$ leaves has $2^{\mathbf{N}-1}+\mathbf{N}-1$ connected sets of vertices.

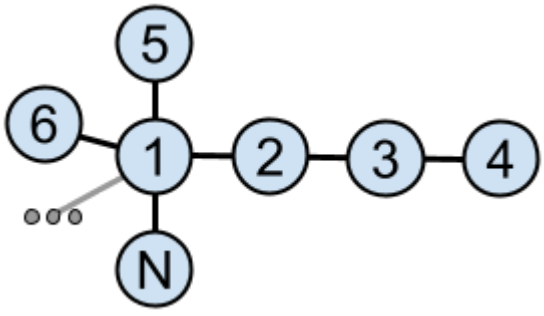
We could go a bit further and notice that isomorphic subtrees can only be considered once, which reduces the number of states much more. We do not know the exact number, but the worst trees we were able to find had between 7 and 8 million non-isomorphic connected subtrees for $\mathbf{N}=40$. This is still quite hard to process within the time limit given that the processing for each state becomes quite expensive, with tree isomorphism involved.

So how could we make things simpler for ourselves?

Test Set 1

One approach is to try to construct a concrete tree by hand that is winning for the second player. The game for this problem was intentionally chosen in such a way that this is not easy: for example, every chain (a tree where each vertex is connected to the previous one, $1-2-3-\cdots-\mathbf{N}$) is winning for the first player, as they can take either 1 or 2 middle vertices and then make symmetric moves.

However, it turns out that for even values of $\mathbf{N}$ there are several relatively simple constructions. For example, consider a starlike tree consisting of the center vertex 1 plus $\mathbf{N}-3$ chains attached to it, one chain with 3 edges ($1-2-3-4$) and all other chains with 1 edge ($1-5, 1-6, \ldots, 1-\mathbf{N}$).



The number of chains with 1 edge is even, so if the first player takes the endpoint of one of them, we can take another one and arrive at a smaller starlike tree with the same property, unless there is just one left, in which case we simply have a single chain with 4 edges and 5 vertices and can win by taking one or three middle vertices of it.

If the first player takes the center vertex together with some neighbors, then we are left with one chain with 2 or 3 vertices, and some number of isolated vertices. We can choose either to eliminate the chain, or to reduce it to an isolated vertex, in such a way that the number of resulting isolated vertices is even and we will win.

If the first player takes two vertices of the longer chain together with the center vertex, then we are left with an odd number of isolated vertices, so we can take one of them and win.

If the first player makes the longer chain shorter, then we can take the center vertex together with some neighbors in such a way that an even number of isolated vertices remains.

If the first player makes a move in the middle of the longer chain such that we have a star plus a separate isolated vertex, then we can again take the center together with some of its neighbors in such a way that an even number of isolated vertices remains.

Finally, if the first player makes a move in the middle of the longer chain such that we have a star plus a chain with 2 vertices, then we remove one of the leaves of the star. If we and the first player keep removing star leaves with each move, then after one of our moves only one leaf will be left, so we will have two chains with 2 vertices each and we can do symmetric moves to win. And if the first player tries taking the center of the star, or do something to the chain with 2 vertices, then we can always get an even number of isolated vertices after our move.

We were not able to come up with a similar explicit construction for odd values of $\mathbf{N}$. Also, even for even values of $\mathbf{N}$ coming up with this construction was not required to solve the problem. You can find an alternative approach in the next section.

Test Set 2

What can we do when $\mathbf{N}$ is odd, or when we cannot come up with the above construction for even $\mathbf{N}$?

The key idea is to consider some very restricted class of trees, so that:

- The trees in the class are diverse enough so that some of them are winning for the second player — as we have seen above, chains are not diverse enough.
- The trees in the class are uniform enough so that we can implement the nim-value computation based on the Sprague-Grundy theorem without having to deal with arbitrary tree isomorphism.
- The number of subtrees of the trees in the class is small enough so that the number of states that the nim-value computation needs to process is small.

We have found that many classes of trees work, for example:

- Starlike trees with at most 5 chains attached to the center vertex.
- Chains with at most two additional chains attached at some points of the main chain.
- Starlike trees with all chains having length 1 except at most 3 (generalizing the Test Set 1 construction).

Having chosen such a class, we can then quickly implement the nim-value computation, and then either do an exhaustive search over all trees in the class, or keep generating random trees in the class until we find at least one example for each value of $\mathbf{N}$ between 30 and 40.

It was also possible to choose a class of trees with enough diversity and fast enough nim-value computation, but that would still require to implement general tree isomorphism. This would of course complicate the implementation a bit more, but still allow to solve the problem. One such class is very narrow trees where vertex i is connected either to vertex $i-1$ or to vertex $i-2$.

Having found the winning tree for each value of $\mathbf{N}$, we can then hardcode them in the solution we submit. We still need the nim-value computation code as part of the solution in order to actually play the game after printing the tree. This was, in fact, one of the reasons for making this problem interactive: had we just required printing a tree for a given $\mathbf{N}$, the submitted

solutions would likely just consist of 10 constants, and it might be possible to just find a solution in a few guesses, at least for Test Set 1.

Of course, it might happen that you choose a class of trees that does not give a solution to all possible cases. However, given the diversity of classes that work in this problem, it is most likely possible to adjust your class slightly in such a way that you do not need to complicate the nim-value code too much, but that allows to find a solution. Of course, doing this quickly required a certain intuition or "feeling" of the problem.

How does the judge work?

While the contestants had the luxury of choosing a class of trees where finding nim-values is quick, they did not have to do it, and therefore the judge had to deal with arbitrary trees.

The judge had a very well-optimized computation of nim-values for the general case, using tree isomorphism to reduce the number of states it has to process. In the worst case we found for $\mathbf{N} = 40$, it could find the nim-values for all states in about 70 seconds (this was optimized about 20x from our first version). However, we were not sure that the case we found is truly the worst (and this was another reason for making this problem interactive), and we also did not want the contestants to have to wait for a long time for the verdict.

Therefore we have introduced additional logic in the judge when considering the very first move of the game. We iterated over possible first moves in the order of increasing size of the largest connected component remaining after the move, and stopped iteration in two cases:

- If we find a move that leads to a losing position. Then we can make this move and win, and do not need to consider other options.
- If we have visited 500000 different states during our search, and have already processed at least one first move fully. Then we stopped processing the current first move and made our solution only choose between the first moves already processed.

The order of checking the first moves ensured that only a few states would need to be visited to check the first option for the first move, because all connected components would have a size of at most 20 after it, therefore we would likely process quite a few first moves before we run out of the 500000 states budget. And for the values of $\mathbf{N}$ up to 33 or so, we would actually process all options for the first move. This allowed us to be relatively confident that we would catch most incorrect solutions, but still have the judge finish in 4 seconds per case.

After the first move, the judge always knew the nim-value exactly and played optimally.

The judge had to make another important decision: when it found itself in a losing position (which is the most important case to consider, as in the other case it could guarantee a win using the nim-values), which move should it do?

Initially the judge would make a random move in such situation. But then we noticed that even for the Test Set 1 solution described above, making random moves might not be enough to catch all bugs, since there are many bugs that would only be caught with a specific first move. Therefore, we have changed the behavior of the judge for choosing the first move: it tried to make such moves that would lead to different situations for the solution to handle after the first move in different games.

Two situations were considered different if they led to non-isomorphic configurations, with the additional step of collapsing $2K + 1$ copies of the same subtree to 1 copy, and $2K$ copies of the same subtree to 2 copies, to avoid the judge wasting the games exploring different ways to remove leaves from a star. After the first move, if the judge was still in a losing position, it would make fully random moves.

Of course, this still left a possibility that some incorrect solutions would win all games against the judge, as it did not visit all possible states — there were at least 7 million states to visit potentially, but the judge had only 50 games, each visiting at most 40 states. However, we hope that the chances of this happening were quite low.